REVA UNIVERSITY
Bengaluru, India

School of Computing Science and Engineering

Course Code: B22EF0601	**Cryptography and Network Security**	**Academic Year: 2026** **Semester & Batch: 6-C2**
Project Details:		
Project Title:	**Diffie-Hellman Key Exchange Simulator**	
Place of Project:	**REVA UNIVERSITY, BENGALURU**	
Student Details:		
Name:	**P A MANNASWINI**	**Sign:**
Mobile No:	**9480339785**	
Email-ID:	**23020104940@reva.edu.in**	
SRN:	**R23EF167**	
Guide Name	**Prof. Shivalingappa**	

ABSTRACT

This project presents an implementation of the Diffie-Hellman Key Exchange algorithm using Python and Streamlit. The main objective is to demonstrate how two users can securely establish a shared secret key over an insecure communication channel without directly transmitting the key.

The system allows users to input parameters such as a prime number, generator, and private keys, and then computes the corresponding public keys and shared secret key. The application visually illustrates that both parties independently generate the same secret key, ensuring secure communication.

This project highlights the importance of public key cryptography and its role in modern security protocols such as HTTPS and VPNs. It provides a simple and interactive way to understand the working of secure key exchange mechanisms.

INDEX

S. No.	Topic	Page No.
1.	Introduction	1
2.	Objective	1
3.	Theory	1
4.	Methodology	2
5.	Implementation	2
6.	Output	3
7.	Advantages	4
8.	Limitations	4
9.	Applications	4
10.	Future Enhancements	4
11.	Conclusion	5
12.	Project Link	5
13.	References	6

1. Introduction

In today's digital world, secure communication is essential for protecting sensitive information transmitted over networks. Cryptography plays a key role in ensuring data confidentiality, integrity, and authentication. One of the major challenges in secure communication is the safe exchange of cryptographic keys between two parties over an insecure channel.

The Diffie-Hellman Key Exchange algorithm provides a solution to this problem by enabling two users to generate a shared secret key without directly transmitting it. This project demonstrates the working of the Diffie-Hellman algorithm through an interactive application built using Python and Streamlit.

2. Objective

- To understand the concept of secure key exchange
- To implement the Diffie-Hellman algorithm using Python
- To develop an interactive interface using Streamlit
- To demonstrate how two parties can generate a shared secret key
- To visualize the process of public and private key generation
- To study real-world applications of cryptography in network security

3. Theory

Diffie-Hellman is a public key cryptographic algorithm used for secure key exchange between two parties over an insecure network.

Key Components:

- **Prime Number (p):** A publicly known large prime number
- **Generator (g):** A primitive root modulo p
- **Private Keys (a, b):** Secret values chosen by users
- **Public Keys (A, B):** Values computed and shared publicly

Mathematical Representation:

- A = g^a mod p
- B = g^b mod p

Shared Secret Key:

- K = B^a mod p
- K = A^b mod p

Both users compute the same key independently due to properties of modular arithmetic.

4. Methodology

1. The user inputs values for:
    1. Prime number (p)
    2. Generator (g)
    3. Private keys (a and b)
2. The system calculates public keys:

    1. A = g^a mod p
    2. B = g^b mod p

3. The public keys are exchanged between users
4. Each user computes the shared secret key:

    1. User A computes: B^a mod p
    2. User B computes: A^b mod p

5. The application verifies that both keys are identical

5. Implementation

The project is implemented using:

- **Python** for core algorithm logic
- **Streamlit** for building the interactive web interface

Modules Used:

- **app.py** → Handles user input and displays output
- **diffie_hellman.py** → Contains the key generation logic

Core Function:

```
def generate_keys(p, g, a, b):
    A = pow(g, a, p)
    B = pow(g, b, p)
    key_A = pow(B, a, p)
    key_B = pow(A, b, p)
    return A, B, key_A, key_B
```

6 . OUTPUT

Figure 1: Diffie-Hellman Key Exchange Simulator – Input and Public Key Generation

Figure 2 : Generation of Public Keys and Shared Secret Key using Diffie-Hellman Algorithm

7. Advantages

- Provides secure key exchange over an insecure channel
- No need to transmit the secret key directly
- Based on strong mathematical principles
- Widely used in real-world security protocols (SSL/TLS)

8. Limitations

- Vulnerable to Man-in-the-Middle (MITM) attacks
- Requires authentication mechanisms for full security
- Computational cost increases with large numbers

9.Applications

- Secure web communication (HTTPS)
- Virtual Private Networks (VPNs)
- Secure messaging systems
- Cryptographic protocols

10. Future Enhancements

- Add MITM attack simulation
- Implement authentication techniques
- Use large prime numbers for better security
- Add encryption using the generated key
- Improve UI with visualization

11.Conclusion

The Diffie-Hellman Key Exchange algorithm is a fundamental technique in cryptography that enables two parties to securely generate a shared secret key over an insecure communication channel. This project successfully demonstrates the working of the algorithm through an interactive application developed using Python and Streamlit.

The results confirm that both users independently compute the same shared key without directly exchanging it, ensuring secure communication. This highlights the importance of public key cryptography in modern network security systems.

Overall, the project provides a clear understanding of secure key exchange mechanisms and their real-world applications, forming a strong foundation for further exploration in cybersecurity and cryptographic protocols.

12. Project Links

The project source code and live application can be accessed through the following links:

- **GitHub Repository:**
  https://github.com/imannaswini/DHK
- **Live Deployed Application:**
  https://deffiehellman.streamlit.app/

These links provide access to the complete implementation and allow users to explore and interact with the Diffie-Hellman Key Exchange Simulator in real time.

13. References

1. William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson Education.
2. GeeksforGeeks, "Diffie-Hellman Key Exchange Algorithm." https://www.geeksforgeeks.org/diffie-hellman-key-exchange/
3. NIST (National Institute of Standards and Technology), Cryptographic Standards and Guidelines. https://www.nist.gov
4. Lecture Notes and Course Materials – Computer Networks and Security (CNS).
5. Streamlit Documentation – https://docs.streamlit.io
6. Python Documentation – https://docs.python.org/3